

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

Μάθημα: Συστήματα Διαχείρισης Μεγάλων Δεδομένων

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ PROJECT

Επεξεργασία Δεδομένων Ροής με Kafka, Spark και MongoDB

Φοιτητής:

Κωνσταντίνος Κολύβρας 1103826

MachineGuid (OS):

ebbd3b1f-289c-480e-83c6-a1dd2d1a1c04

Υπεύθυνοι Καθηγητές:

Β. Μεγαλοοικονόμου

Ε. Ζαχαράκη

1 Περιγραφή Αρχιτεκτονικής

Η εφαρμογή υλοποιεί μια real-time stream processing pipeline με τέσσερα επίπεδα:

- **Data Producer (producer.py):** Ο UXSIM traffic simulator παράγει δεδομένα κίνησης οχημάτων. Τα αποτελέσματα φιλτράρονται (μόνο οχήματα με $v > 0$) και αποστέλλονται ως JSON records στο Redpanda topic `vehicle_positions` μέσω της βιβλιοθήκης `kafka-python`.
- **Message Broker (Redpanda):** Το Redpanda είναι ένας broker συμβατός με το Kafka API, ο οποίος λαμβάνει και αποθηκεύει προσωρινά τα μηνύματα. Εκτελείται μέσω Docker και εκθέτει δύο listeners: εσωτερικά στο `redpanda:9092` (για το Spark) και εξωτερικά στο `localhost:19092` (για τον producer).
- **Stream Processing (spark_streaming.py):** Το Apache Spark Structured Streaming καταναλώνει τα δεδομένα από το topic, ορίζει schema για το parsing των JSON μηνυμάτων και υπολογίζει στατιστικά (`vcount`, `vspeed`) ανά time και link με `groupBy`.
- **NoSQL Database (MongoDB):** Αποθηκεύει τα raw data στο collection `raw_data` και τα επεξεργασμένα stats στο collection `stats`, και τα δύο στη βάση `traffic`.

2 Σχεδιαστικές Επιλογές

- **Δίκτυο UXSIM:** Δημιουργήθηκε δίκτυο 4x4 grid με 4 intersections (I1-I4) και traffic signals με κύκλο 60 δευτερολέπτων. Η τυχαία ζήτηση ορίζεται κάθε 30 δευτερόλεπτα. Η εξαγωγή γίνεται με τη `vehicles_to_pandas()` και επιστρέφει τα πεδία: `t`, `veh`, `link`, `x`, `v`.
- **Spark Schema:** Ορίστηκε StructType schema για το parsing των JSON μηνυμάτων, εξασφαλίζοντας σωστούς τύπους (DoubleType για `t`, `x`, `s`, `v` και StringType για `link`, `orig`, `dest`).
- **MongoDB Sinks:** Τα raw data γράφονται με `outputMode("append")`. Τα stats γράφονται μέσω `foreachBatch`, καθώς το update mode με aggregations δεν υποστηρίζεται απευθείας από τον MongoDB Spark connector.

3 Προβλήματα που Αντιμετωπίστηκαν και Λύσεις

1. **Numpy JSON Serialization:** Οι τύποι `np.int64` και `np.float64` του UXSIM δεν γίνονται serialize από την τυπική βιβλιοθήκη `json` της Python. Υλοποιήθηκε η συνάρτηση `convert(obj)` που μετατρέπει τις τιμές σε native int/float πριν την αποστολή.
2. **Φιλτράρισμα Κινούμενων Οχημάτων:** Η εκφώνηση ζητά αποστολή μόνο οχημάτων σε κίνηση. Προστέθηκε έλεγχος $v > 0$ στον producer πριν το `send()`, ώστε να αποκλείονται τα σταματημένα οχήματα.
3. **Docker Networking:** Ο Spark consumer τρέχει σε Docker container ενώ ο producer τρέχει στο host OS. Ρυθμίστηκαν διπλοί advertised listeners στο Redpanda (`redpanda:9092` εσωτερικά, `localhost:19092` εξωτερικά).

4 Screenshots και Αποτελέσματα

4.1 Redpanda: Λήψη Μηνυμάτων στο Topic

```
PS C:\Ceid\BigData\Project spring semester 2026\Code> docker exec -it redpanda rpk topic consume vehicle_positions -n 5
{"topic": "vehicle_positions",
"value": "{\"name\": \"8\\\", \"dn\": 5, \"orig\": \"E1\\\", \"dest\": \"W1\\\", \"t\": 5, \"link\": \"E1I4\\\", \"x\": 0.0, \"s\": -1.0, \"v\": 50.0}\",
"timestamp": 1781457357192,
"partition": 0,
"offset": 1322235
}
{"topic": "vehicle_positions",
"value": "{\"name\": \"16\\\", \"dn\": 5, \"orig\": \"W1\\\", \"dest\": \"W1\\\", \"t\": 5, \"link\": \"W1I1\\\", \"x\": 0.0, \"s\": -1.0, \"v\": 30.0}\",
"timestamp": 1781457357193,
"partition": 0,
"offset": 1322236
}
{"topic": "vehicle_positions",
"value": "{\"name\": \"24\\\", \"dn\": 5, \"orig\": \"S2\\\", \"dest\": \"W1\\\", \"t\": 5, \"link\": \"S2I2\\\", \"x\": 0.0, \"s\": -1.0, \"v\": 30.0}\",
"timestamp": 1781457357193,
"partition": 0,
"offset": 1322237
}
{"topic": "vehicle_positions",
"value": "{\"name\": \"32\\\", \"dn\": 5, \"orig\": \"N3\\\", \"dest\": \"W1\\\", \"t\": 5, \"link\": \"N3I3\\\", \"x\": 0.0, \"s\": -1.0, \"v\": 30.0}\",
"timestamp": 1781457357193,
"partition": 0,
"offset": 1322238
}
{"topic": "vehicle_positions",
"value": "{\"name\": \"40\\\", \"dn\": 5, \"orig\": \"S4\\\", \"dest\": \"W1\\\", \"t\": 5, \"link\": \"S4I4\\\", \"x\": 0.0, \"s\": -1.0, \"v\": 30.0}\",
"timestamp": 1781457357193,
"partition": 0,
"offset": 1322239
}
```

Σχήμα 1: Λήψη JSON μηνυμάτων στο Redpanda topic `vehicle_positions` μέσω `rpk topic consume`.

4.2 Spark: Αποτελέσματα Streaming Console

Batch: 9				
time	link	vcount	vspeed	
130.0 I140	1		50.0	
130.0 I4I3	2	25.833333333333336		
125.0 S4I4	4		25.0	
130.0 I242	1		30.0	
130.0 N3I3	3	21.666666666666668		
130.0 S4I4	3	21.666666666666668		
125.0 N1I1	4		25.0	
130.0 S2I2	3	21.666666666666668		
130.0 N1I1	3	21.666666666666668		
125.0 S2I2	4		25.0	
125.0 I3I2	2	38.333333333333336		
125.0 I4I3	2		45.0	
130.0 I2I1	1		30.0	
130.0 I3S3	1		30.0	
125.0 N3I3	4		25.0	
130.0 E1I4	1		35.0	
125.0 I2I1	2	38.333333333333336		
130.0 I1S1	1		30.0	
125.0 I141	2		45.0	
125.0 E1I4	2		42.5	
only showing top 20 rows				
Batch: 10				
time	link	vcount	vspeed	
135.0 I242	1		30.0	
135.0 I4I3	2	21.666666666666664		
135.0 S4I4	2		30.0	
135.0 I3I2	1	33.33333333333333		
135.0 N1I1	2		30.0	
135.0 I141	1	33.33333333333333		
135.0 I3S3	1		30.0	
135.0 S2I2	2		30.0	
135.0 I2I1	1	33.33333333333333		
135.0 N3I3	2		30.0	
135.0 I1S1	1		30.0	
135.0 I4I4	1		30.0	

Σχήμα 2: Micro-batches από τον Spark Structured Streaming consumer, με στατιστικά (time, link, vcount, vspeed) ανά ακμή.

4.3 MongoDB: Collections και Documents

```
test> use traffic
...
switched to db traffic
traffic> show collections
...
raw_data
stats
traffic> █
```

Σχήμα 3: Τα collections `raw_data` και `stats` στη βάση `traffic` της MongoDB.

```
traffic> db.raw_data.findOne()
...
{
  _id: ObjectId('6a05e657fcedb92e96c3c27b'),
  name: '4',
  dn: 5,
  orig: 'E1',
  dest: 'W1',
  t: 10,
  link: 'E1I4',
  x: 0,
  s: -1,
  v: 50
}
traffic> █
```

Σχήμα 4: Document από το collection `raw_data`.

```
traffic> db.getCollection('stats').findOne()
...
{
  _id: ObjectId('6a05e658fcedb92e96c3c281'),
  time: 10,
  link: 'S4I4',
  vcount: Long('1'),
  vspeed: 30
}
traffic> █
```

Σχήμα 5: Document από το collection `stats`.

5 Aggregation Queries στη MongoDB

5.1 Query 1: Μέση Ταχύτητα ανά Link

```
1 db.getCollection('stats').aggregate([
2   { $group: { _id: "$link",
3               avg_speed: { $avg: "$vspeed" },
4               total_records: { $sum: "$vcount" } } },
5   { $sort: { avg_speed: -1 } } ])
```

```

    %%% [id, total_records] = get_collection('data').aggregate([
        {
            '$group': {
                '_id': '$title',
                'avg_speed': { '$avg': '$speed' },
                'total_records': { '$sum': '$count' }
            }
        },
        {
            '$sort': { avg_speed: -1 }
        }
    ])

    %%%
    %%% 'id',
    avg_speed: 45.7030646411897,
    total_records: long( 1048 )

    %%%
    %%% 'id', avg_speed: 36, total_records: long( 442 )
    %%%
    %%% 'id', avg_speed: 36, total_records: long( 381 )
    %%%
    %%% 'id', avg_speed: 36, total_records: long( 22 )
    %%%
    %%% 'id', avg_speed: 36, total_records: long( 190 )

    %%%
    %%% 'id',
    avg_speed: 10.9595771206554,
    total_records: long( 1068 )

    %%%
    %%% 'id',
    avg_speed: 9.71637708050547,
    total_records: long( 1376 )

    %%%
    %%% 'id',
    avg_speed: 9.51519805609708,
    total_records: long( 1139 )

    %%%
    %%% 'id',
    avg_speed: 9.2928167821731,
    total_records: long( 1151 )

    %%%
    %%% 'id',
    avg_speed: 8.57304813774806,
    total_records: long( 1939 )

```

Αποτελέσματα Query 1.

5.2 Query 2: Οι 5 Πιο Συμφορη- μένες Ακμές

```
1 db.getCollection('stats').aggregate([
2   { $group: { _id: "$link",
3               avg_density: { $avg: "$vcount"
4                             },
5               max_vehicles: { $max: "$vcount"
6                             } } },
7   { $sort: { avg_density: -1 } },
8   { $limit: 5 }
9 ])
```

```

traffic: {
  get_location: {
    status: 'aggregate'
  },
  {
    $sort: {
      id: "3104a",
      avg_vehicle_density: { $avg: "percent" },
      max_vehicle_count: { $max: "percent" }
    }
  },
  {
    $sort: { avg_vehicle_density: -1 }
  },
  {
    $limit: 5
  },
  {
    $id: "1"
  },
  {
    {
      $id: "waiting_at_origin_node",
      avg_vehicle_density: 87.402534231669,
      max_vehicle_count: 100
    },
    {
      $id: "1413",
      avg_vehicle_density: 21.26731562939162,
      max_vehicle_count: "68"
    },
    {
      $id: "1414",
      avg_vehicle_density: 20.332812116662,
      max_vehicle_count: "68"
    },
    {
      $id: "1416",
      avg_vehicle_density: 20.332812116662,
      max_vehicle_count: "68"
    },
    {
      $id: "1417",
      avg_vehicle_density: 18.23326,
      max_vehicle_count: "55"
    },
    {
      $id: "1418",
      avg_vehicle_density: 17.927084470404,
      max_vehicle_count: "68"
    }
  }
}

```

Αποτελέσματα Query 2.

6 Προαιρετικό (Bonus): REST API με FastAPI

Υλοποιήθηκε απλό REST API (αρχείο `api.py`) με τη βιβλιοθήκη FastAPI, το οποίο διαβάζει δεδομένα απευθείας από τη MongoDB και εκθέτει τα παρακάτω endpoints:

- GET /stats — Επιστρέφει τα τελευταία aggregated stats (time, link, vcount, vspeed).
- GET /stats/top5 — Επιστρέφει τις 5 πιο συμφορημένες ακμές (κατά μέσο αριθμό οχημάτων), μέσω MongoDB aggregation pipeline.
- GET /stats/avg-speed — Επιστρέφει τη μέση ταχύτητα ανά ακμή σε όλο το διάστημα εξομοίωσης.
- GET /raw — Επιστρέφει τα τελευταία raw αρχεία θέσης οχημάτων.

Το API εκτελείται ως ξεχωριστό service στο Docker Compose (`traffic-api`, port 8000) και συνδέεται στη MongoDB μέσω της μεταβλητής περιβάλλοντος `MONGO_URI`.

Προσβάσιμο στη διεύθυνση <http://localhost:8000/docs> (αυτόματο Swagger UI).

7 Συμπέρασμα

H pipeline λειτούργησε επιτυχώς από άκρο σε άκρο. Ο UXSIM simulator παρήγαγε ρεαλιστικά δεδομένα κυκλοφορίας, τα οποία ο Spark Structured Streaming consumer κατανάλωσε σε real-time, υπολόγισε στατιστικά ανά ακμή και τα αποθήκευσε στη MongoDB. Τα aggregation queries αναδεικνύουν τις πιο συμφορημένες ακμές του δικτύου. Επιπλέον, υλοποιήθηκε REST API (bonus) που εκθέτει τα δεδομένα της MongoDB μέσω HTTP endpoints.